



Turbo Rascal Syntax Error

Turbo Rascal Syntax Error, “;” expected but “BEGIN”

MANUAL

TRSE (or its full original name “Turbo Rascal Syntax error, “;” expected but “BEGIN”) is a complete suite (IDE, compiler, programming language, resource editor) intended for developing games/demos for 8 / 16-bit line of computers, with a focus on the MOS 6502, the Motorola 68000, the (GB)Z80, the M6809 and the X86. TRSE currently supports application development for the C64, C128, VIC-20, PLUS4, NES, Gameboy, PET, ZX Spectrum, TIKI 100, Amstrad CPC 464, Atari 2600, 8086AT, Amiga 500, Atari 800, BBC Micro, Super Nintendo (SNES), Mega65, VZ200, MSX, Apple II, Commander X16, TIM-011, Videoton TVC, TRS80 CoCo3, the Vectrex, Thomson (mo5 and mo6), Atari ST 520 AND MORE!

CONTENTS:

Getting Started.....	3
The GUI.....	4
The GUI with an open Project.....	5
Starting a new Project.....	6
Useful functionality.....	8
Size Analyser.....	8
Memory Map.....	8
Program structure.....	9
Compression.....	9
Chars.....	9
Variables.....	10
Arrays.....	11
User-defined types.....	11
Array of arrays/strings.....	12
Pointers and References.....	12
Units.....	13
Comments.....	15
Functions and Procedures.....	15
Conditional Statements.....	16
Loops.....	17
Inline assembly code.....	18
Inline assembly unrolling.....	18
Preprocessors.....	19
Constants.....	19
Preprocessor ifdef / if / else / endif.....	19
Pre-existing preprocessor defines.....	19
Other preprocessor commands.....	20
Records.....	21
Classes.....	22
Platform VZ200.....	23

Getting Started

Download TRSE from LemonSpawn:

<https://lemonspawn.com/turbo-rascal-syntax-error-expected-but-begin/downloads/>

There is a nightly build with new updates. You do not necessarily need to update nightly as some updates may be minor, but updating monthly is recommended.

Extract the files to a location on your drive you want to run TRSE from. TRSE does not have a SETUP.EXE Just extract the files. To run it, doubleclick on the TRSE or TRSE.exe application to launch.

More information:

<https://lemonspawn.com/turbo-rascal-syntax-error-expected-but-begin/setup/>

The main TRSE webpage has links to Tutorials, videos, TRSE Syntax and more:

<https://lemonspawn.com/turbo-rascal-syntax-error-expected-but-begin/>

In a nutshell, Turbo Rascal Syntax error, “;” expected but “BEGIN” is a complete suite for developing games and demos for older computer systems. TRSE is created with Qt (C++), and runs as a stand-alone application that contains various tools for developing and deploying projects for these processors.

There are builds for Windows, Mac OSX and Linux.

TRSE is written using QT for cross-platform availability.

<https://www.qt.io/>

There is a Showcase with videos of Projects created by TRSE developers:

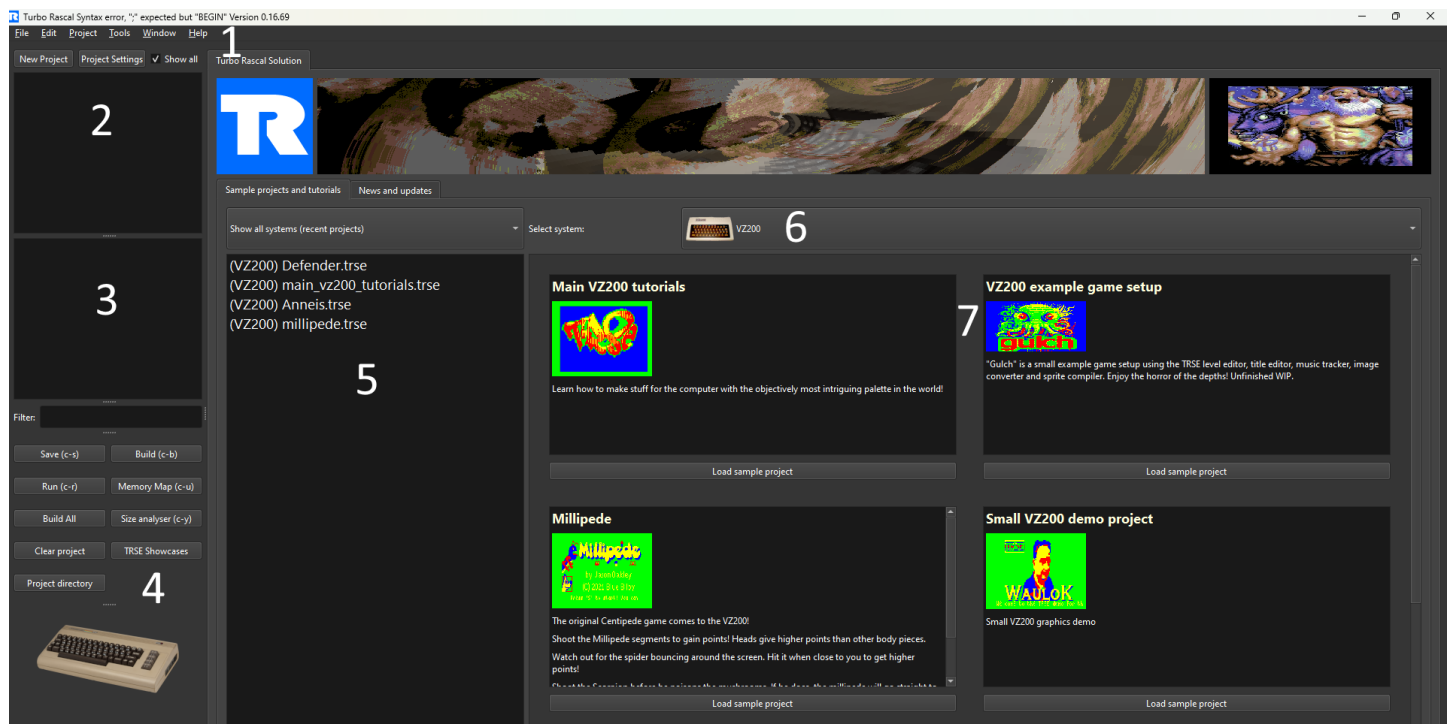
<https://lemonspawn.com/gallery/>

This shows what TRSE can really do!

What can you make with TRSE?

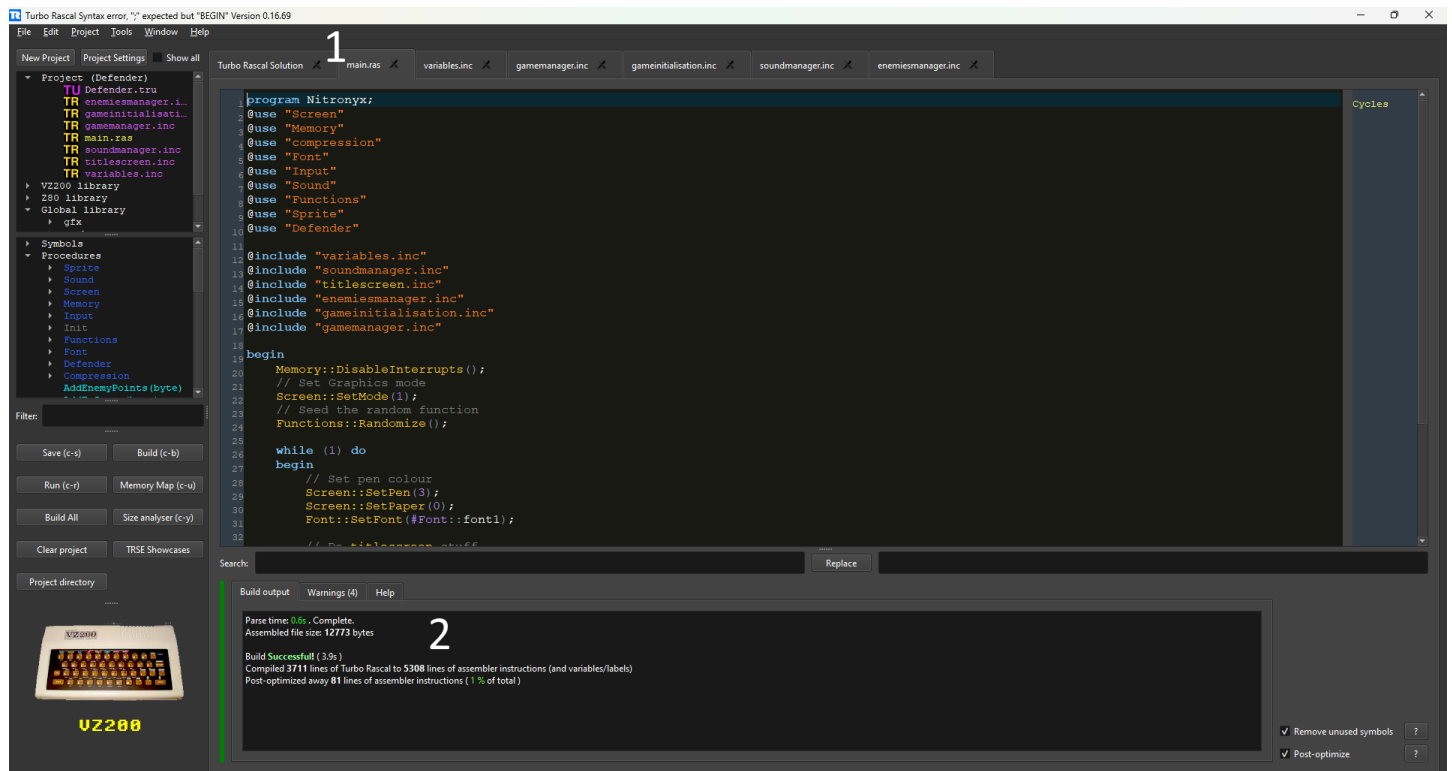
Just about anything your imagination can think of. Games, Demos, Tools, Utilities. The list is endless. TRSE compiles your Pascal-like code into fast-running Assembly code. It's easier to code with than 100% Assembly but just as fast to execute!

The GUI



1. Menus. This contains useful functionality for the GUI such as build, load, search.
2. Project Explorer. This shows the files and folders used in the current project.
3. Functions and Procedures. After you first compile your Project, this displays Functions and Procedures. You can click on them to jump to the file and position they are in.
4. Buttons. Shortcut buttons for common menu items. Also shows a picture of the current platform.
5. Project List. This lists recently opened Projects.
6. Platform Selector. Select the computer platform you will be developing on.
7. Example Projects. Lists example tutorials and applications which come with TRSE to help you learn developing on your selected platform.

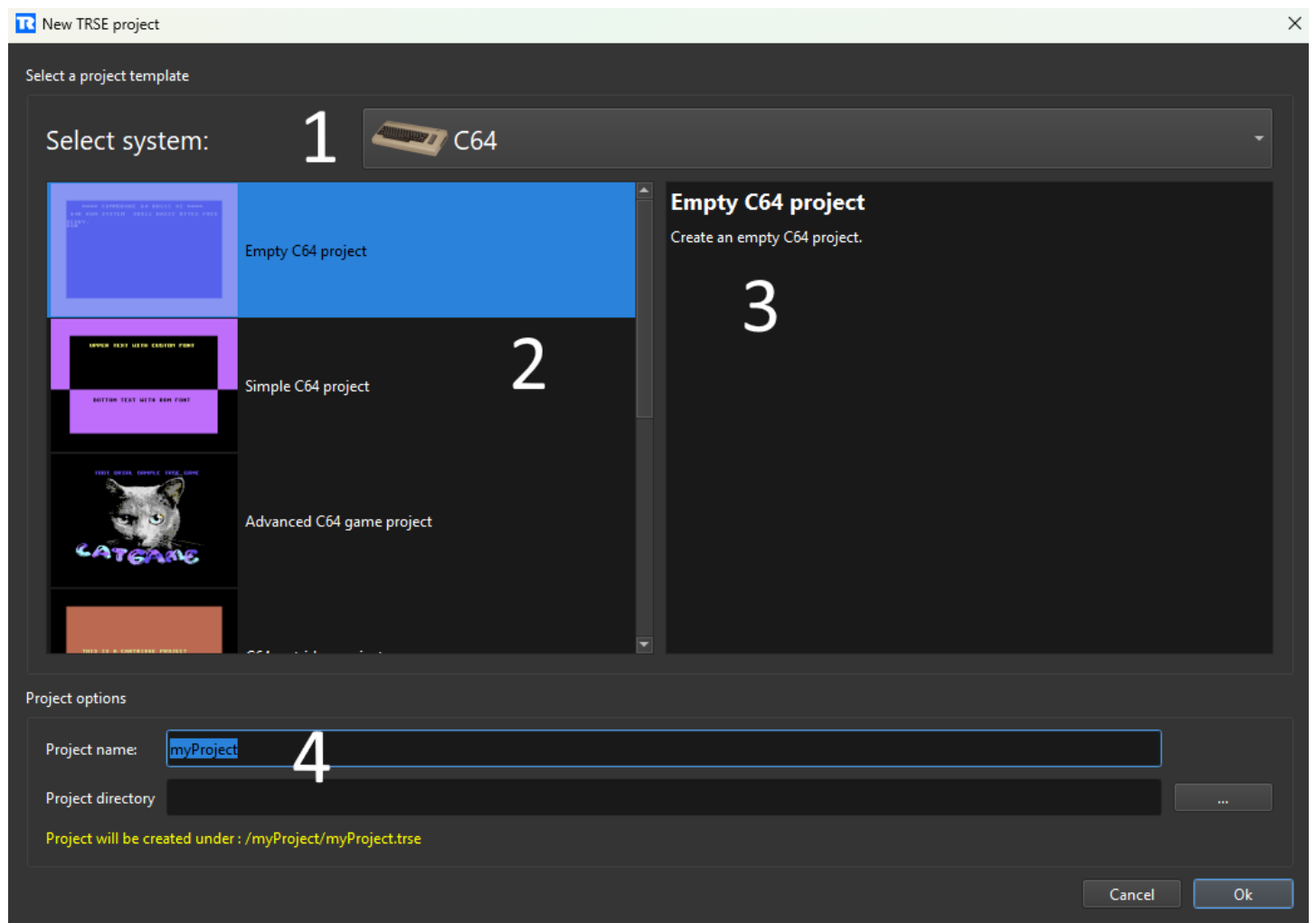
The GUI with an open Project



Additional GUI areas:

1. Tabs. Currently opened files appear as tabs for easy switching between files while editing.
2. Build Output. The output of building your Project will show compiling status and highlight errors.

Starting a new Project



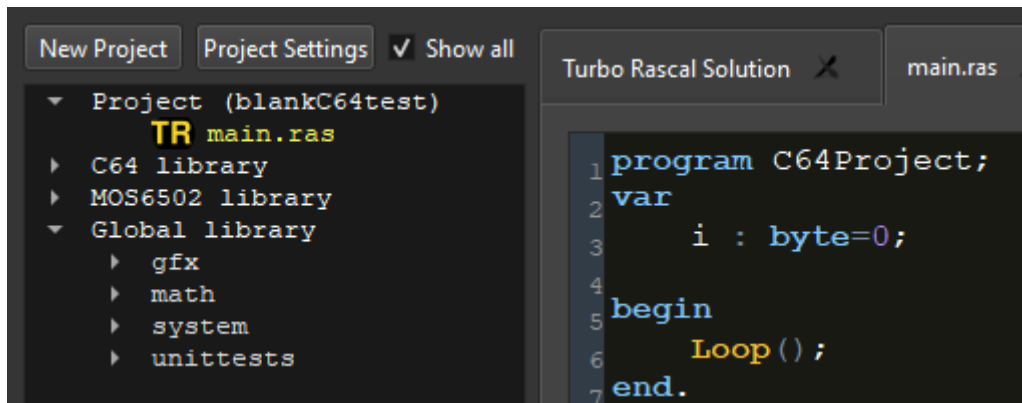
1. Platform Selector. Select the system you will develop on.
2. Templates. Select a Template to begin your Project.
3. Description. Shows information about the Template selected.
4. Naming. Type your Project name and select the directory to store your Project.

Press OK to create the Project.

An example. "Empty C64 project" will create a new folder using your Project Name in the selected directory and enter a blank template ready for you to code.

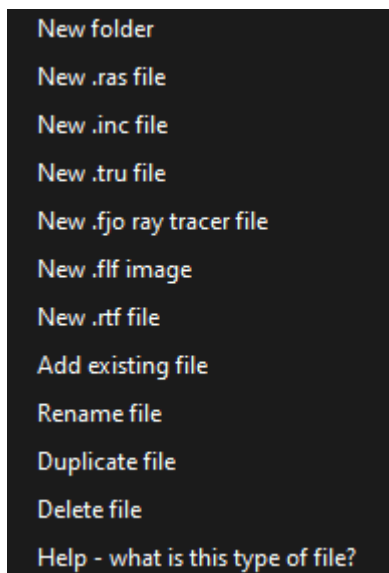
It's highly recommended to add this Project directory to a GIT setup so you can synchronize your program files to your Github.

The empty project is created:



Your code starts after the 'begin' clause.

The main file is always called *main.ras*. You can right-click on the line **Project (projectName)** and get a menu to add other files:



Additional files for your Project are usually **.inc** files. Eg. *playerinput.inc* *enemymanagement.inc*. **.tru** files are for TRSE Units. These are used across projects and there will be more information on this later. You can also add *fjo*, *flf* and *rtf* files. The Help will tell you about the file type selected.

It's recommended to have different files for different functionality.
Eg: *initialisegame.inc* *soundsmanager.inc* *enemies.inc*

You can add these to your project easily:

```
@include "initialisegame.inc"  
@include "soundsmanager.inc"  
@include "enemies.inc"
```

This makes your code easier to read and work with.

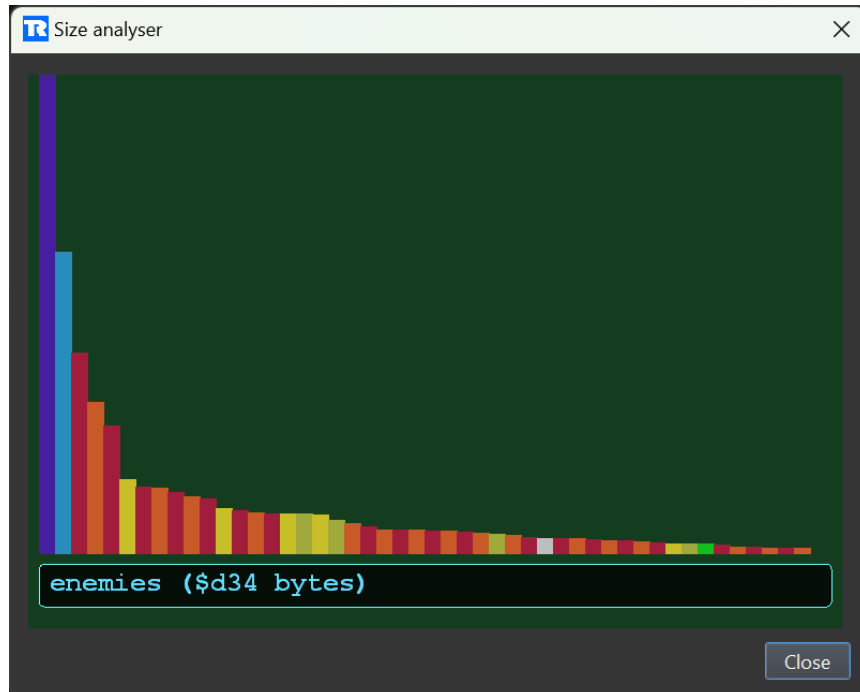
Useful functionality

TRSE has useful built-in functions.

Size Analyser

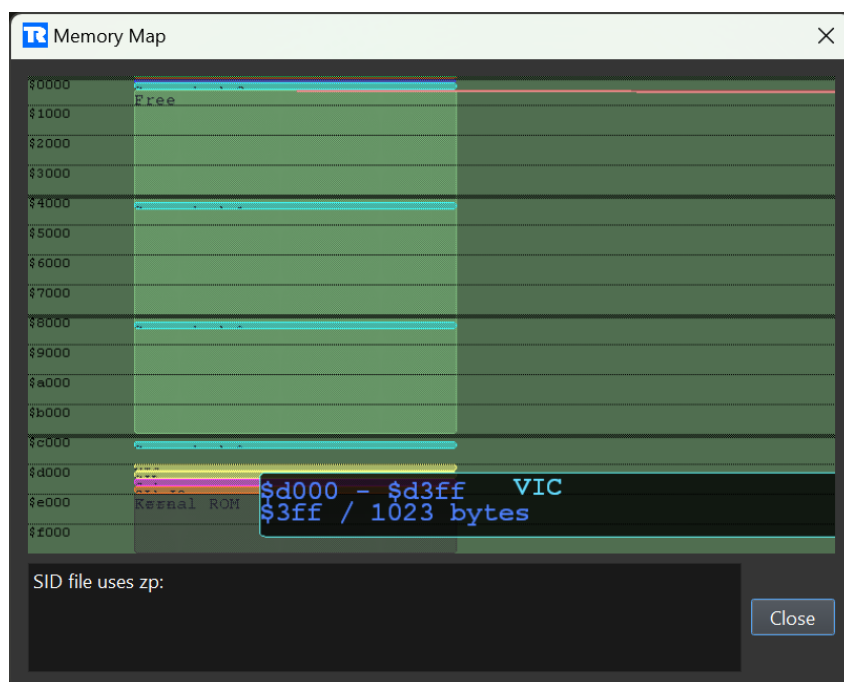
After Building your code at least once.

Pressing the *Size analyser* button or pressing CTRL-Y:



This analyses your code and displays a graph showing which part takes the most space in your program. You can use this to work out any parts which need optimising or separating out into more Functions or Procedures.

Memory Map



This shows a memory map of the platform you are developing on. Hovering over each section displays a tool-tip with information about that section of memory.

Program structure

Programs must start with the “program” keyword. Variable declarations can happen anywhere after a var is defined. Procedures and functions are defined by their respective keywords. The main block starts with a “begin” and “end”, as do code blocks.

```
Program MyProgram;
var
    a,b,c : byte = 0;

Procedure Myprocedure(mp_i:byte);
var
    somProcedureVariable : byte;
begin
    // Do someting
end;

// This is the main block.
begin
    // Call user-defined procedures etc
    MyProcedure(2);
end.
```

Compression

On systems using the MOS6502, Motorola 68000, Z80 or the Motorola 6809 you can automatically LZ4-compress data using the “compressed” flag. Use the “compression” library to deflate your data:

```
@use "compression/compression"
var
    image : incbin("mydata.bin") compressed;

begin
    Compression::Decompress(#image,#videomemory);
```

Variables

Each platform has a generic set of variables types, but some have limitations.

You declare your variables at the start of your Project. It's generally easier on low-resource Platforms to make the variables global.

TRSE supports several data types:

- byte : values 0-255. Supported on all systems.
- integer: values 0-65535. Supported on all systems.
- long : 32-bit double word. Supported on all x86 and m68k systems. On the 6502, "long" is a 24-bit number.
- pointer : Points to an address in memory. Uses zero pages on the 6502, "long" on the x86/m68k and is simply an integer on the (GB)Z80
- lpointer: 24-bit pointers for the 65c816.
- boolean: Syntactically identical to a byte.
- string : An array of characters. Syntactically identical to an array of bytes.
- incbin : include a binary file
- incsid : include SID music file (C64 only)
- incnsf : include NSF music file (NES only)

Declaring variables:

I usually set up a file called *variables.inc* This way I can easily find their definitions and add/remove as needed.

An example of declaring variables:

```
var
    x, y : byte;
    temp : Integer;
    message : string = "Press 'S' or 'Fire' to start!";
    lookupSpriteY : array[64] of byte = BuildTable("i/4+1");
    myp : pointer;
    spritesData: array[] of byte =(000, 000, 000, 000, 000, 052, 002, 072, 000, 064, 228);
    const numEnemies : byte = 20;

    player_record = record
        x : byte;
        y : byte;
        dead : boolean;
    end;
    playerShip : player_record;
```

Since TRSE is Pascal-based you assign variables using :=

ie.

```
x := 2;
```

Being a Pascal-like language, TRSE requires a semi-colon at the end of most lines (with a few exceptions).

Often when compiling you may miss a semicolon and you will see an error in the output. The error is often:

Error during parsing in file : gamemanager.inc

Fatal error on line: 433

Did you forget a semicolon? (Token should not be ID in Parser)

Hence the “; expected but begin”.

Arrays

TRSE supports arrays of base types (which includes addresses of symbols), records and classes.

If the number of elements in the array doesn't match the declared count, TRSE will pad the missing items with "0".

If you don't know the number of elements in the array – but have a list of data – you can declare the count to be missing by using [].

```
// Preprocessor constants are basically string-replace operation performed before compile
@define somePreprocessorConstant 25

var
  // Real constant
  const someConstant : byte = 25;
  myArray1 : array[128] of integer; // 128*2 bytes of zeros are declared
  myArray2 : array[4*someConstant] of byte = (0,5,$90, someConstant*2);
  myArray3 : array[] of integer = ($1000, $2000, $3000, someConstant*$100); // unspecified array count
  myArray4 : array[255-55] of byte = (1,2,3); // only 3 values defined, rest will be padded with zeros.
  myMonsters : array[10] of Monster; // "Monster" is either a record or class
  // Buildtable fills arrays compile-time with user-defined javascript code (i is a counter here ranging from 0-256)
  sin : array[256] of byte = BuildTable("Math.sin(i/256.0*2*3.14159)*127 +128");
  sin2 : array[256] of byte = BuildSineTable(127); // how handy!
  // Buildtable2d (indices i and j) lets you generate a 2D table:
  const height : byte = 64;
  const width : byte = 64;
  circles : array[] of byte =
  buildtable2d(height,width,"256/(Math.sqrt(Math.pow(i-WIDTH/2,2)/2+Math.pow(j-HEIGHT/2,2))+4)*8");
```

Arrays are accessed as such:

```
myArray[i]:=someValue*5;
```

Note: there is no out-of bounds testing in TRSE.

On the 6502 you cannot have arrays of pointers, as they must reside in the zeropage address space. Instead, use an array of integers, and simply assign a pointer value to the integer array element in order to use it, i.e.

```
var
  addresses : array[ 10 ] of integer = (#sym1, $2020, ... )
  zp : pointer;
zp := addresses[i];
zp[0] := ...
```

User-defined types

It is possible to define your own types in TRSE:

```
type mat4 = array[16] of integer;
rotMat : mat4;
```

Array of arrays/strings

String lists in TRSE are declared as a list of pointers containing the addresses of the declared strings.

Example:

```
var
  strlst : array[ 3 ] of string = ("HELLO", ("YOU",10,65), "WORLD");
  zp : pointer; // a pointer

printstring(strings[1],0,40); // will print out "HELLO"
strlst[3]:="NEW STRING";
```

Pointers and References

Pointers are the back-bone of any programming language, and are simply variables that contain (“points to”) a memory address. A TRSE pointer p can point to any of the three standard built-in types : byte, integer or long, where the corresponding lookup p[index] automatically adjusts for 1,2 or 4 byte data lengths. Pointers can also point to class objects, but not to records (since these are laid out non-chunky in memory by TRSE).

The reference symbol in TRSE is #, analogous to C's &. # returns the address of a variable/data, and can be assigned to pointers – or 16/32 bit integers/longs. Example:

```
var
  myArray[512] of byte = BuildTable(" i*2 "); // builds a table with values 0,2,4,6,8 .....
  someText : string="SOME TEXT";
  p: pointer; // default byte pointer, same as Pascal ^byte or "pointer of byte"
  ip : pointer of integer; // integer pointer, same as Pascal ^integer
  ip2 : ^integer; // same as "pointer of integer"
begin
  p:=#myArray; // p now points to myArray
  p[2] := 10; // same as myArray[2] := 10;
  p:=#myArray + i*8; // p now points to myArray + i*8
  p:=#someText + 3; // points to the 3rd letter in the string
  ip := #myIntegerArray; // ip will access elements as integers (16-bit, or 2 byte numbers)
```

Chars

A character is a single byte represented with a single letter encased in ". 'A' and 65 are treated as identical in TRSE.

```
myByte := 'C';           // same as myByte:=67;
if (myByte='5') then ...
```

Units

TRSE Units are files with code specific to each platform.
These add Functions and Procedures you can use for your project.

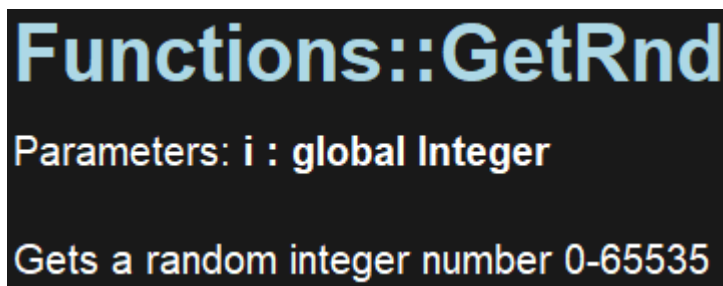
An example might be for easily drawing text to the screen or drawing and moving a sprite or making a sound effect.

Each platform has its own set of Units – often written in direct Assembly code – in order to make your life easier.

You usually do not need to edit any TRU Unit file. This is done by a developer on that platform and is updated when the next nightly release of TRSE is published.

If you wish to add to the Units on a current or new platform, please contact **Leuat** via the Lemonspawn website to give you access to the TRSE GIT.

If you wish to know how to use a particular Units Function or Procedure, type it in and hit F1. If the Units developer included instructions it will pop up. Eg:



The image shows a screenshot of a function help window. The title is 'Functions::GetRnd' in a large, bold, blue font. Below the title, the parameters are listed as 'Parameters: i : global Integer' in a smaller, bold, yellow font. At the bottom, the description 'Gets a random integer number 0-65535' is written in a bold, yellow font. The background is black.

Units can be placed in 4 different levels, depending on the usage:

- Project level: The .tru file is located somewhere in your project folder, and can only be accessed from your project. Example: game monster library, inventory library
- System level: The .tru file (located in TRSE/units/[SYSTEM]) can be accessed from all projects for the given system (ie C64, Amstrad, X86, Amiga). These libraries should typically contain system-specific functions such as screen setup, text output, line drawing, file handling etc.
- CPU level: The .tru file (located in TRSE/units/cpu_specific/[CPU]) can be accessed from all projects for any system that uses the specific CPU (ie 6502, Z80, M68K, X86 etc). These libraries should contain system-independent methods that are useful for all projects of the same CPU. Examples include: compression/decompression, math methods (matrix operations, vectors, operations etc)
- Global level: the .tru file (located in TRSE/units/global) is accessible for *all* systems using *all* CPU types. This means that the code is completely platform independent. Examples include: string manipulation libraries, 3d engine

It is possible to mix levels of units, where global units contain “high level code” and system-specific units contain “low-level” code (perhaps with direct asm + system-specific calls). As an example, think of a wireframe 3D renderer:

- A global unit containing high-level methods such as mesh creation, draw calls, sorting methods, list of 3d objects etc. Accessible from all systems.
- CPU-specific units containing mathematical matrix/vector operations. These units are used in the main global 3d engine. (example: 6502 matrix class, z80 matrix class etc)
- A system-specific graphics class for line drawing: when the “global” unit calls the respective “draw line” method from the graphics unit, the actual implementation is performed on a system level (since a drawline method would vary greatly between the C64, BBC Micro and Amiga 500).

Declaration:

```
Unit MyUnit;
var
  var1, var2 : byte;
  const ANumber : integer = 200;
  Monster = record
    x,y,health : byte;
  end;

// keyword "global" reuses var1
procedure Proc1(var1 : global byte);
begin
  // .. do some stuff
end;

end.
```

Usage:

```
Program myProgram;
@use "MyUnit"
var
  monsters : array[ MyUnit::ANumber ] of MyUnit::Monster;
  MyUnit::Proc1( 10 );
```

Note that you access unit variable/methods by “Unit::var”, where “::” is the default separator. “::” is also synonymous to “_”, so you can also access the members by “Unit_var” instead. Using “_” is mandatory when accessing variables/methods within assembly blocks.

Comments

Comments are specified using the double-slash or slash-star-star methods.

```
// This code gets player input from keyboard or joystick
```

```
/** This other code does a lot of stuff  
    So we use slash-star-star across multiple lines **/
```

Functions and Procedures

Functions and Procedures are code blocks you can call within your program. Functions can return values whereas Procedures do not. Also, code blocks are between 'begin' and 'end;' statements.

TRSE currently only supports 'byte', 'integer' and 'boolean' as return values.

An example Procedure:

```
// Update score  
procedure UpdateScore();  
begin  
    Screen::SetPen(1);  
    DrawPadded(score,6,0);  
end;
```

You can call this simply: UpdateScore();

Passing values to Procedures:

```
// Update score  
procedure AddToScore(p : global byte);  
begin  
    gameStats.score := gameStats.score + p;  
    UpdateScore();  
end;
```

An example Function returning a byte-sized value:

```
function GetPen():byte;  
begin  
    asm("  
        ld a,(#7846)  
        ld [Screen_c],a  
    ");  
    GetPen := c;  
end;
```

You can call this simply: pencolour := GetPen();

The line

```
GetPen := c;
```

returns the value in the 'c' variable of the current Unit from the function .

As you can see, you can also use inline Assembly.

You can forward declare a function by using the "forward" keyword:

```
function Calc( p : byte ) : byte; forward;
```

Conditional Statements

TRSE offers the standard if, then, else conditionals.

Eg:

```
if (x < 40) then x := x -1;
```

Nested conditionals are specified using brackets.

Eg:

```
if (x < 40 and (y > 12)) then SoundEffect(1);
```

Conditionals with code blocks

To add code blocks for your conditionals and when using the 'else' clause.

Eg:

```
if (x < 20) then
  begin
    y := y + 2;
  end
else
  begin
    y := y + 4;
  end;
end;
```

While conditionals works similar to if statements:

```
while (a>0) do
begin
  PrintSomething(a);
  dec(a);
end;
```

Cases tests a single variable for multiple values as such:

```
case i of
  0: DoSomething();
  1: begin
    HelloThere();
    b:=b+5;
  end;
  2+i : ThisIsWeird();
else
begin
  PerformElseBlock();
end;
```

The 'else' code block is like C's 'default'.

Note: Beware that each section should not be too large and use Functions or Procedures as it is easy on 8bit platforms to exceed the relative jump range.

Loops

TRSE has the standard for, to, step loops:

```
for w := 0 to 100 step 10 do
begin
    Sound::Play(200+w,1);
    Sound::Play(180,1);
end;
```

Step is optional.

You can specify the compiler to unroll your loops for faster execution, at the cost of larger code:

```
for i:=0 to 5 unroll do
begin
    doSomething(i+1);
end;
```

```
// syntactically the same as
doSomething(1);
doSomething(2);
doSomething(3);
doSomething(4);
doSomething(5);
```

Control Statements

You can control your loops with *continue* and *break*.

```
for i:=0 to 10 do
    if (i=5) then continue; // if i=5 then the program jumps back to the start of the for loop where i is now 6
    if (i=8) then break; // breaks off the loop when i=8
end;
```

Inline assembly code

TRSE allows the user to insert assembly code anywhere in a statement block. You can mix between Pascal and asm at will.

```
for i:=0 to 5 do
begin
  v += 5;
  asm("
    lda v
    asl
    tax
myLoop:
    // etc
");
end;
```

Note: Since variables are internally sometimes stored with prefixes (such as unit variables etc), you will have to be careful when accessing these variables from an inline assembly block. Variables with the same name as internal CPU registers are always prefixed (such as “hl”, “a”, “bc” on the z80 and “ax”, “bx” etc on the x86). Z80 example:

```
Unit SomeUnit;
var
  screen : ^byte;
...
// screen variable is internally stored as "SomeUnit_screen"
asm("
  ld hl, [ SomeUnit_screen ] ;
```

Inline assembly unrolling

```
asm("
  repeat 50
  lda $8000 + [i*8]
  sta $4400 + [i]
  repend
");
/*
```

The above code will generate the following unrolled code:

```
lda $8000 + 0
sta $4400 + 0
lda $8000 + 8
sta $4400 + 1
lda $8000 + 16
sta $4400 + 2
...
*/
```

Preprocessors

TRSE has a lot of preprocessor directives defined. These can be used to give messages to the compiler before the actual parsing of the program is executed.

Constants

Constants are preprocessors that will be string replaced before compilation occurs.

```
@define myConstant 4
#define myText "WHOO PARTY"
a:=@myConstant * 5; // same as 4*5
```

Preprocessor ifdef / if / else / endif

You can define preprocessor constants and use them in preprocessor expressions as such:

```
@define L 1
#define N 10

#ifdef L // If @L is defined...
    @if N = 10 // if @N holds the value of 10
        screen_bg_col:=1;
    @else
        screen_bg_col:=2;
    @endif
@else // if L is not defined
    screen_bg_col:=4;
#endif
```

Pre-existing preprocessor defines

```
#ifdef C64
// the following line will only be compiled if the current project is for the C64
screen_bg_col := black;
#endif

#ifdef CPU_Z80
// the following line will only be compiled if the current project is for a Z80 CPU
asm(" push bc ");
#endif
```

The syntax for cpu-specific defines are CPU_MOS6502, CPU_Z80, CPU_PX86, CPU_M68K, CPU_GBZ80 and CPU_M6809

Other preprocessor commands

TRSE contains a lot of preprocessor commands that can perform a variety of function. Some examples are:

```
@raiseerror "This text will halt compilation and display this message"
```

```
@raisewarning "This text will continue compilation and display this message as a warning"
```

```
@deletefile "myfile.bin"
```

```
// will export 10 sprites from mysprites.flf
```

```
// @export works on all .flf files, and will export to the system's
```

```
// native video format
```

```
@export "mysprites.flf" "mysprites.bin" 10
```

```
// will export a subregion in a file. Only works on select systems
```

```
@exportsubregion "tiles.flf" "tiles.bin" 0 0 8 8
```

```
// Sprite compiler will convert sprites to pure .asm functions for that extra speed. Only works on select systems.
```

```
@spritecompiler "images/sprites1.flf" "tank" 3 0 16 8
```

There are a lot of undocumented built-in preprocessor commands for automatically building noise data, path tools, conversion, file handling, sprite compiling etc.

Records

A record is similar to a “typedef” in C, and lets the programmer bundle a set of variables together to a primitive object. TRSE allows for creation of these objects as arrays. Internally, each element of the record (or array) is stored as individual arrays. This allows for fast lookup on the 6502 and z80 etc, but prevents any pointers to records as the data isn’t bundled together.

Since records are internally stored as arrays of data, having arrays within records is not possible. You will need to define a class instead, which will store data sequentially instead of a distributed manner.

```
program SomeRecords;
var
    monster_type = record
        x,y : byte; // Current position of monster
        color : byte; // Current colour of monster
        health : byte;
    end;

    player : monster_type; // Declare "player" as a single monster type
    const noMonsters : byte = 10;
    // Declare an array of monsters_type
    monsters : array[ noMonsters ] of monster_type;
```

Classes

A class is similar to a Record, but has a couple major distinctions. First, you can define methods (procedures) within the class that operates on its properties. Also, class data is stored sequentially – or chunky – as opposed to records. This makes lookup a bit slower on the 6502, but you can now have pointers to the objects. Third, it is possible to have arrays and strings located inside classes (as opposed to records).

Use the “this” pointer in order to access local class variables within a method.

```
var
  Monster = class
    x,y : byte; // Current position of monster
    color : byte; // Current colour of monster
    health : byte;
    procedure Draw();
    begin
      moveto(this.x, this.y,hi(screen_char_loc));
      screenmemory[0] := 0; // draw a @
    end;
  end;

  player : Monster; // Declare "player" as a single monster type
  const noMonsters : byte = 10;
  // Declare an array of monsters_type
  monsters : array[noMonsters] of Monster;
  pm : pointer of Monster;
```

```
pm := #monsters[3]; // Point to the 3rd monster
pm.x := 10;
pm.Draw();
monsters[3].Draw(); // same as pm.Draw();
```

Please note: Classes are still in its very early implementation, and do not yet support inheritance/constructors/private/public properties etc. Also, in order to ensure good coding practice (= I haven't managed to write an optimizer for this yet), setting properties directly of arrays of objects is not allowed:

```
monsters[i].x := 10; // NOT allowed in TRSE
monsters[i].y := 20; // NOT allowed in TRSE
```

These calls would have to calculate $i * \text{sizeof}(\text{Monster})$ on every step and set a temporary pointer to the memory location. Instead, you'll have to do this manually – enforcing good coding standards:

```
var
  pm : pointer of Monster;

pm := #monsters[i]; // Points to the i'th element
pm.x := 10;
pm.y := 20;
```

Platform VZ200

This section has information related to developing for the Dick Smith VZ200 Z80 platform.

Font Unit

The VZ200 platform has a unit defined for drawing text and numbers in mode(1) graphics format. A font has been provided, although you could always create your own.

To add to your project use:

```
@use "Font"
```

First, make sure you are in the graphics mode using:

```
@use "Screen"
```

```
Screen::SetMode(1);
```

Set the pen colour and paper colour and configure the font to draw:

```
Screen::SetPen(3);
```

```
Screen::SetPaper(0);
```

```
Font::SetFont(#Font::font1);
```

This sets the pen to colour 3, which is Red.

Colours available with background (Paper) 0: 0=Green, 1=Yellow, 2=Blue, 3=Red.

Paper is the background mode. Options are 0=Green, 1=Buff.

Colours available with background (Paper) 1: 0=Buff, 1=Cyan, 2=Magenta, 3=Orange.

To draw text on screen:

```
Font::DrawTextAt("Score",0,0,false);
```

This draws the text "Score" at x,y positions 0,0. If 'true' was specified it would draw non-destructively using XOR which preserves any existing pixels but changes their colour. If you drew the exact same text again at this location it would invert and restore the previous existing pixels. 'false' means to draw and destroy existing pixels.

To draw numbers defined in a variable you first need to convert the data to a string:

```
Functions::ByteToString(gameStats.lives);
```

```
Font::DrawTextAt(#Functions_s,17,0,false);
```

The 's' variable in the 'Functions' unit holds the string once created. You can then use Font::DrawTextAt with the pointer to the string as #Functions_s

Screen Unit

The Screen Unit is for functionality in mode(0) text mode and mode(1) graphics mode.

To clear the screen in mode 0 or mode 1:

```
Screen::Cls(0); // mode(0)
```

```
Screen::Cls(1); // mode(1)
```

You can get the current pen colour:

```
pencolour := Screen::GetPen();
```

To get the colour of a pixel at a specific x,y location:

```
colour := Screen::GetPixel(5,5);
```

To draw a pixel in the current pen colour to the specified x,y coordinates:

```
Screen::PutPixel(4,4);
```

To draw a line in the current pen colour from x1,y1 to x2,y2:

```
Screen::DrawLine(10,10,20,20);
```

To draw a line in the specified pen colour from x1,y1 to x2,y2. Colour 2=Blue:

```
Screen::DrawLineColor(10,10,20,20,2);
```

To draw a rectangle in the current pen colour, specify the starting x,y coordinates with width and height:

```
Screen::DrawRect(5,5,10,10);
```

To draw a filled rectangle in the current pen colour, specify the starting x,y coords and provide a width & height:

```
Screen::DrawRectFilled(5,5,10,10);
```

To wait for the raster Vertical Blank. This is useful for gaming to synchronise the speed of drawing to the screen:

```
Screen::WaitForVerticalBlank();
```

To get and set bytes in memory, particularly on the screen:

```
Screen::Poke(28672,12);
```

```
data := Screen::Peek(28672);
```

There is built-in code for scrolling data on the screen:

```
Screen::doSmoothScroll(28672, 5);
```

This is a 1-pixel smooth scroll left on screen beginning at 'pos' with a screen width and the height of 'height'.

There is a faster, coarser scroll by 4 pixels at a time left on screen (or in memory) beginning at '28672' with a screen width and the height of '5':

```
Screen::doCoarseScroll(28672, 5);
```

You can also scroll left or right by 1-pixel with wrap-around:

```
Screen::doSmoothScrollWrapL(28672,5);
```

```
Screen::doSmoothScrollWrapR(28672+31,5);
```

You can also scroll left or right by 4-pixel with wrap-around:

```
Screen::doCoarseScrollWrapL(28672,5);
```

```
Screen::doCoarseScrollWrapR(28672+31,5);
```

To scroll the screen up one line at a time starting at 28864 and scrolling 57 lines height:

```
Screen::doLineScroll(28864, 57);
```

Input Unit

This allows you to get input from a keyboard or joystick.

Enable with:

```
@use "Input"
```

To get input from the keyboard, one key at a time:

```
key := Input::ReadKey();
```

If no key pressed, 0 is returned. Otherwise the code for this key is returned. This uses the built-in ROM routine.

Another, similar one was created for MPAGD:

```
key := Input::GetPressedKey();
```

You can use this to wait for a specific key to be pressed:

```
if (Input::GetPressedKey() <> Input::Key_S) then
```

The key values are present in the Input Unit file.

To wait for a keypress and not return until a value is received:

```
key := Input::WaitChar();
```

These are based on the VZ200 keymap:

: ROW :	BIT POSITION					:
: ADDRESS :	5	4	3	2	1	0 :

: 68FEH :	R	Q	E		W	T :
: 68FDH :	F	A	D	CTRL	S	G :
: 68FBH :	V	Z	C	SHFT	X	B :
: 68F7H :	4	1	3		2	5 :
: 68EFH :	M	SPC	,		.	N :
: 68DFH :	7	0	8	-	9	6 :
: 68BFH :	U	P	I	RETN	O	Y :
: 687FH :	J	;	K	:	L	H :

You can also read them independently for faster use in games:

```
s := Screen::Peek(26863);
```

Memory location 26863 is 68EF in Hexadecimal. You can check for the Spacebar being pressed with:

```
@use "Functions"
```

```
if (Functions::IsBitSet(s,16) = false) then // Spacebar pressed
```

The Functions Unit is detailed on the next page.

Functions Unit

This provides various useful functions.

We covered `IsBitSet()` in the previous page. There is also:

```
Functions::SetBit(flags,16);  
Functions::ResetBit(flags,16);
```

These set or reset bits in a byte variable.

To get a random Integer using the built-in ROM routines:

```
intVar := Functions::GetRnd(1000);  
If you prefer to get a byte-sized value, you can convert it:  
byteVar := Functions::IntToByte(Functions::GetRnd(31));
```

You can also convert a byte to an Integer:

```
intVar := Functions::ByteToInt(byteVar);
```

To reseed the randomizer function:

```
Functions::Randomize();
```

Another way to get a random number is to use an existing array of randomised bytes:

```
r := Functions::getRandFromArray(#arr);
```

To convert a number value to a string:

```
stringVar := Functions::IntegerToString(intVar);
```

The same with a byte:

```
stringVar := Functions::ByteToString(byteVar);
```

To find the byte size of a string (up to 255 chars):

```
byteVar := Functions::SizeOfString(stringVar);
```

To find the index value of the last character in a string:

```
byteVar := Functions::LastIndexOfChar(stringVar);
```

To get the absolute value of a variable:

```
absVal := Functions::abs(byteVar);
```

Memory Unit

Activate with:

```
@use "Memory"
```

To enable or disable interrupts:

```
Memory::EnableInterrupts();  
Memory::DisableInterrupts();
```

You can also save or restore all registers quickly:

```
Memory::PushAll();  
Memory::PopAll();
```

Sound Unit

Provides access to make sound effects.

Activate with:

@use "Sound"

To make the built-in VZ200 keyboard BEEP sound:

Sound::Beep();

To play a sound at a specific frequency and duration:

Sound::Play(freq,dur);

Both are integer sized.

To play a shooting sound effect (like a gun):

Sound::Shoot(freq,dur);

To play an explosion sound (takes some time and stops processing other things, so use wisely):

Sound::Explode();

Sprite Unit

Provides the ability to draw sprites directly to the screen. Note the sprites are in VZ 4-bits-per-pixel format.

I recommend downloading **Aseprite** to design your sprites. Each sprite is a new Frame in the Aseprite image you are drawing. Export them using the lua script I created for VZ200:

<https://github.com/WauloK/VZ200-Aseprite-Export>

This has a built-in option to export a sprite array for TRSE.

Define your sprite as a byte array:

spriteData: array[] of byte =(010, 128, 010, 192, 009, 080, 001, 000, 007, 064, 007, 064, 001, 000, 003, 192);

Configure the sprite data for TRSE. The 2 stands for 2x4 pixels so this sprite size is 8x8 pixels:

Sprite::SetData(#spriteData);

Sprite::SetSize(2,8);

You can change the SetData and SetSize on the fly so that you can have different sized sprites with different data arrays defined.

Draw the sprite at the coordinates:

Sprite::DrawAt(10,20,0,false);

This draws sprite frame number 0 (zero) at 10,20 and not preserving the pixels beneath. 'true' would preserve.

Text Unit

This lets you draw to the mode(0) text-only mode.

Activate with:

@use "Text"

Printing a character to the screen. Note: control chars are obeyed and not printed.

```
Text::PutChar(33);    // Draw a hash # on the screen at the current cursor position.
```

To print a character or string to the screen at the current cursor position.:

```
Text::Print("TESTING");
```

To print a character or string inversely to the screen at the current cursor position.:

```
Text::PrintInv("INVERSE TEXT");
```

Moving the cursor to the top-left of the screen:

```
Text::CursorHome();
```

To move the cursor to a specific location:

```
Text::MoveCursorTo(10,5);
```

To print a number to the screen. Useful for score variables, etc:

```
Text::PrintNumber(intVar);
```

To quickly print a string at coordinates:

```
Text::PrintAt(0,2,"TEST",0);
```

The last parameter is 0 for normal text and 1 for inverse.

To print a number at a specific coordinate:

```
Text::PrintNumberAt(0,2,intVar,0);
```

The last parameter is 0 for normal text and 1 for inverse.